Transaction Management is one of the most important concepts in real-time software development—especially in **banking, e-commerce, ticket booking, UPI/payments, ERP systems**, etc.

Below is a **detailed + real-world explanation** of Transaction Management, including **when to use what**, with practical examples (Java/Spring style).

---

1) What is a Transaction?

A **transaction** is a group of operations that must be executed as **one unit**.

✅ Example: Bank Transfer

1. Debit money from A
2. Credit money to B
3. Save transfer history

If **any step fails**, **everything must rollback** (undo).

That's a transaction.

---

2) Why Transaction Management is Needed?

Without transaction management:

- Debit succeeds ✅
- Credit fails ❌
- Result → Money disappears □ (data inconsistency)

So transaction management ensures:

✅ Consistency
✅ Atomicity
✅ Rollback on failure
✅ Locking control (to avoid dirty data)

---

3) ACID Properties (Real-time explanation)

ACID	Meaning	Real Time Example
A - Atomicity	All or nothing	Debit & credit must both happen, else rollback
C - Consistency	Data rules always valid	Balance should never become negative
I - Isolation	Transactions don't interfere	Two people booking the last seat at same time
D - Durability	Once committed, stays saved	Even if server crashes, committed payment stays saved

---

4) Types of Transaction Management

✅ (A) Programmatic Transaction Management

You manually manage commit/rollback in code.

Example (JDBC)

```
Connection con = dataSource.getConnection();
try {
   con.setAutoCommit(false);

   debitAccount(con, "A", 5000);
   creditAccount(con, "B", 5000);

   con.commit();
} catch(Exception e) {
   con.rollback();
} finally {
   con.close();
}
```

When to use Programmatic?

✅ Best for:

- Legacy apps (pure JDBC)
- Very customized transaction logic
- When you want full manual control

❌ Not recommended in Spring apps because it increases boilerplate.

✅ (B) Declarative Transaction Management (Most used)

Transaction is managed using **annotations/config**, not manual commit/rollback.

Example (Spring Boot)

```
@Transactional
public void transferMoney(Long fromAcc, Long toAcc, double amount) {
   accountRepo.debit(fromAcc, amount);
   accountRepo.credit(toAcc, amount);
}
```

When to use Declarative?

✅ Best for:

- 95% real-world Spring applications
- Service-layer transaction control
- Clean & maintainable code

---

5) Where should transactions be applied? (Most important real-time rule)

✅ **Apply transactions at Service Layer**, not Controller.

Correct

```
@Service
public class PaymentService {
   @Transactional
   public void pay() { ... }
}
```

❌Wrong

```
@RestController
public class PaymentController {
   @Transactional // avoid
   public void pay() { ... }
}
```

Reason: Controllers should handle HTTP only, not DB atomicity.

---

6) Transaction Propagation (WHEN TO USE WHAT)

Propagation defines **what happens when a transactional method calls another transactional method**.

✅ Propagation Types Table (Real-time usage)

Propagation	Meaning	Real-time use case
REQUIRED (default)	Join existing tx, else create new	Normal service calls
REQUIRES_NEW	Always start new tx	Audit logs must commit even if main fails
NESTED	Savepoint inside tx	Partial rollback needed (rare, advanced)
SUPPORTS	If tx exists use it, else no tx	Read-only optional operations
NOT_SUPPORTED	Suspend tx, run non-tx	Heavy reporting query
MANDATORY	Must already have tx	Called method must not run alone
NEVER	Must not run inside tx	Some special integrations

✅ REQUIRED (default) — Most common

Example: normal business flow

```
@Transactional
public void placeOrder() {
   saveOrder();
   reduceInventory();
   makePayment();
}
```

Everything runs in same transaction.

✅ Use when all steps must rollback together.

✅ REQUIRES_NEW — must commit separately

Example: Audit logs / login history

Even if payment fails, audit should be saved.

```
@Transactional
public void paymentFlow() {
   paymentRepo.debit();

   auditService.logAudit(); // REQUIRES_NEW

   throw new RuntimeException("fail");
}
```

Audit service:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logAudit() {
   auditRepo.save(...);
}
```

✅ Use when:

- audit logs
- error logs
- retry queue save
- notification history

✅ NOT_SUPPORTED — no transaction

Example: Reporting / heavy read query

```
@Transactional(propagation = Propagation.NOT_SUPPORTED)
public List<Report> generateReport() {
   return reportRepo.findBigReport();
}
```

✅ Use when:

- long running read queries
- you don’t want locks
- avoid holding DB transaction too long

---

7) Isolation Levels (WHEN TO USE WHAT)

Isolation decides **how much transactions can see each other's uncommitted data**.

Common Issues:

1. Dirty Read

Tx2 reads data updated by Tx1 but Tx1 rolls back.

2. Non-repeatable Read

Tx1 reads data twice but gets different results due to update by Tx2.

3. Phantom Read

Tx1 fetches rows, Tx2 inserts new rows, Tx1 reads again and sees new rows.

Isolation Table

Isolation	Prevents	Use case
READ_UNCOMMITTED	nothing	almost never
READ_COMMITTED (common default)	dirty read	normal apps, payments, CRUD
REPEATABLE_READ	non-repeatable	inventory check
SERIALIZABLE	phantom read	seat booking / extreme safety

✅ **Example: Seat booking**

You want maximum consistency.

```
@Transactional(isolation = Isolation.SERIALIZABLE)
public void bookSeat(Long seatId) {
   Seat seat = seatRepo.findById(seatId).get();
   if(seat.isBooked()) throw new RuntimeException("Already booked");
   seat.setBooked(true);
}
```

✅ Use SERIALIZABLE only when:

- seat booking
- stock trading order placement
- critical unique allocation

Because it's expensive (slow + locks).

---

8) Rollback Rules (Real-time confusion)

By default Spring rolls back only for:

✅ RuntimeException and Error
❌ Not for checked exceptions (Exception)

Real-time example

```
@Transactional(rollbackFor = Exception.class)
public void process() throws Exception {
  // if checked exception occurs, rollback will happen now
}
```

✅ Use rollbackFor when:

- your service throws checked exceptions
- business exception must rollback

---

9) Read-only transactions (performance tuning)

For read operations:

```
@Transactional(readOnly = true)
public User getUser(Long id) {
   return userRepo.findById(id).orElseThrow();
}
```

✅ Use when:

- only SELECT queries
- improves performance (flush disabled / optimization)

---

10) Real-world Scenarios (Big picture)

✅ Banking / UPI

Use:

- REQUIRED for debit+credit
- REQUIRES_NEW for audit logs
- SERIALIZABLE only for critical allocation

✅ E-Commerce Checkout

Use:

- Transaction for order + inventory update
- Payment call should be separate (external gateway is not transactional)

✅ Ticket Booking

Use:

- SERIALIZABLE or locking
- sometimes NESTED/savepoints

✅ HR/ERP Payroll

Use:

- REQUIRES_NEW for logs
- REQUIRED for payroll calculation+save

---

11) Common mistakes in real projects

❌ Keeping transaction open while calling external APIs
Example:

```
@Transactional
public void pay() {
   saveOrder();
   callRazorpay(); // ❌don't do inside transaction
   savePayment();
}
```

✅ Better approach:

1. Save order (TX)

2. Call payment gateway (No TX)
3. Save payment result (TX)

---

12) Best Practices (Industry standard)

✅ Use @Transactional at **service methods**
✅ Keep transactions short
✅ Avoid external API calls inside transaction
✅ Use REQUIRES_NEW for audit logging
✅ Use readOnly=true for reads
✅ Use higher isolation only when required
✅ Prefer optimistic locking for concurrency (JPA versioning)

If you want, I can provide a **complete Spring Boot project** with:

✅ Order Service + Payment Service
✅ Propagation examples
✅ Isolation examples
✅ rollbackFor examples
✅ real DB (MySQL/PostgreSQL)
✅ logs showing commit/rollback

Just say: **"Give full Spring Boot project for transaction management"**.